□定Q&A — WireGuard for Apache NuttX (Community Over Code, Glasgow)

coc-glasgow-slides.html(□ーネル移植後、2026-10-12 の簡素化版、39枚)用の Q&A。`presentation-script.md` の□定**Q&A**は古い `slides.html`(**apps**版のみ、 **FLAT**ビルド前提)向けで、この版には合わない — 特に「 PROTECTED/KERNELは 今後の課題」という回答はもう事実と異なる(検証済み)。このデッキで話す/ 質問を受けるときはこちらを使うこと。

各項目は「JAで理解する内容」+「実際に英語で言う一言」。根拠は verification-matrix.md、 handoff.md、 tai64n-decision.md、 open-questions.md。「全部検証済み」「マージ可能」とは絶対に言わない — 検証マトリクスの書き方に合わせ、できたことと同じ重さでできていないことも言う。

設計・移植方針

Q. なぜ **lwIP** をそのまま **NuttX** に載せなかったのか**?** NuttX は独自のネットワークスタックを持っていて、そちらが標準です。 lwIP を 別途載せると二重にスタックを持つことになり、メモリ的にも保守的にも不利です。 NuttX ネイティブの netdev として実装すれば `ifconfig` や既存のアプリが そのまま使えます。 *Say:* "NuttX has its own network stack; adding lwIP alongside it means two stacks in memory and in maintenance. A native netdev gets you `ifconfig` and every existing app for free."

Q. `apps/` 版とカーネル版、結局どちらが「本物」なのか**?** 両方とも本物で、目的が違います。apps版(v0.1.1、FLATビルド専用)は `psock_*()` という内部APIに依存していて、これは同一アドレス空間の FLATビルドでしか使えません。□ーネル版は `drivers/net/wireguard/` に 住む正式な in-tree ドライバで、PROTECTED/KERNELビルドでも動きます。 提出候補として小さな単位に分けてあるのは□ーネル版です。apps版は「動くと分かっていたものを 土台に、どこを直せば □ーネルに収まるか」を洗い出すために先に作った、 という位置づけです。 *Say:* "The apps version proved the protocol port and found four bugs on a FLAT build. The kernel driver is the one split into review-sized pieces — it works under every privilege model, not just FLAT."

Q. 内部 **API (`psock_*()`)** を使うのは問題にならないのか**?** apps版ではそれが制約でした (FLATビルド専用)。カーネル版はもうこれに 依存していません — `wg0` は□ーネル内の lower-half netdev で、ソケットも 受信スレッドもドライバ自身が所有し、ユーザー空間とは固定長・ポインタなしの ioctl ABI でやり取りします。これが KERNEL ビルドと PROTECTED ビルドの MPU 分割の下で動く理由です(PROTECTED で見つかった間欠的な失敗は `wg` のスタック不足で、直しました。QEMU のみ)。 *Say:* "That was true of the apps version. The kernel driver doesn't touch internal socket APIs at all — it owns the socket itself and talks to user space through a flat, pointer-free ioctl ABI, which is why it runs under KERNEL builds and with the MPU split of PROTECTED builds (an intermittent fault there was a too-small wg stack, and it is fixed, in QEMU)."

Q. カーネルに **BSD-3** のコードを入れるのか**?** なぜ **Apache-2.0** で書き直さないのか**?** はい、入ります。プロトコル本体の `drivers/net/wireguard/wg_noise.c` / `wg_noise.h` は wireguard-lwip(BSD-3-Clause、Daniel Hope)の `wireguard.c` / `wireguard.h` から派生した コードで、ファイル先頭の SPDX にそう書き、NuttX の `LICENSE` にも項目を足しています。 そのため `NET_WIREGUARD` は `ALLOW_BSD_COMPONENTS` に依存します。暗号(BLAKE2s、 ChaCha20-Poly1305、Curve25519)は NuttX 自身の `crypto/` を使っていて、コピーした ものではありません。クリーンルームでの書き直しは可能ですが、出自を正直に書いて おく方を選びました。どちらがよいかは、まだ決まっていません。 *Say:* "Yes. The protocol

core, wg_noise.c, is derived from wireguard-lwip under BSD-3-Clause. It says so in the SPDX header and in the LICENSE file, and the driver depends on ALLOW_BSD_COMPONENTS. The crypto is NuttX's own. A clean-room rewrite is possible, but I chose to keep the attribution honest. Which way is better is not decided."

Q. マージされた後、誰が保守するのか**?** 決まっていません。約束できないことは言いません。今言える事実は、ドライバが NuttX の既存の部品(netdev lower half、`crypto/`、UDP ソケット)の上に作ってあり、外のコードに追従し続ける必要がある部分は小さい、ということです。 *Say:* "That is not decided, and I don't want to promise what I can't. What I can say is that the driver is built on NuttX's own pieces — the netdev lower half, crypto/ and a UDP socket — so there is little outside code to follow."

Q. IPv6 は使えるのか**?** 今は IPv4 だけです。□ーネル版の Kconfig は `depends on NET_UDP && NET_IPv4` で、トンネルの中も外も IPv4 です。IPv6 には対応していません。 *Say:* "IPv4 only, inside and outside the tunnel. IPv6 is not supported."

Q. NuttX の **UDP** は `SO_RCVTIMEO` を実装しているのでは**?(**スライド**21**の一覧表、バグ **1/6)** はい、実装しています(`net/udp/udp_recvfrom.c` が `s_rcvtimeo` で待つ)。バグ 1/6(detach された pthread)は、今は一覧表の1行とノートだけで話していますが、これはその「最初の症状」です: `recvfrom()` が戻らないのを見てこのオプションのせいだと判断し、`poll()` に替えましたが、それでも何も送られず、本当の原因はスレッドが `wg` コマンドと一緒に消えていたことでした。死んだスレッドも「`recvfrom()` が戻らない」ように見えます。`SO_RCVTIMEO` を使う古い経路は、その後試し直していません。 *Say:* "Yes, it does. That was only my first guess for bug one: I blamed the option, but the real cause was that the thread was gone. I have not re-tested the old path since."

検証の範囲**(**スライド**29**〜**31**の深掘り。読み上げから移した細部を含む**)**

Q. 「どこで何が動いたか」**(**スライド**29)**で、実機で足りていないものは何か**?** ESP32-S3 での資源実測(スタック高水位など)と、`esp32s3-devkit:knsh`(実シリコン上のPROTECTEDビルド)。今回計測できたのは SPRESENSE (Cortex-M4F)側です。資源実測はスクリプト化してありますが、まだ回せていません。 PROTECTED は試しました: ビルドは通りますが、手元の CoreS3 は bootloader を使う形式のイメージを FLAT でも起動せず、UART0 が外に出ていないので起動ログも読めません。WireGuard とは関係しない問題で、確かめるには UART0 が出ている DevKitC が要ります。 ESP32-S3自体でのトンネル動作(ネイティブWi-Fi、4/4成功)は検証済みで、2026-10-12 には提出するシリーズそのものを CoreS3 の FLAT ビルドで動かしました(Linux とハンドシェイク、両方向の ping、最大 MTU まで。記録)。 *Say:* "ESP32-S3 resource measurement and PROTECTED on that board's real silicon. The measurement is scripted but has not run yet. I tried PROTECTED: it builds, but my CoreS3 does not boot that kind of image even as FLAT, and it has no UART to read the boot log. That is not a WireGuard problem; a DevKitC with UART0 is needed. The tunnel itself on ESP32-S3 is verified, and on October twelfth the exact submitted series ran there in a FLAT build."

Q. PROTECTED のメモリ配置は特別に調整したのか**?(**スライド**29)** いいえ、rv-virt の protected 構成の既定のままです。□ーネル側の `.text` は 209,832 B、ユーザー側は 21,576 B で、それぞれ 256 KB のうち□ーネルが約8割、ユーザーコマンドが約8% を使っています。繰り返し実行での例外(以前のイメージで8回中2回、2026-10-12 のイメージでは10回中7〜9回。ユーザー空間の `nxsem_wait`、タスク `wg`)はメモリ配置ではなく `wg` のスタック不足が原因で (システムコールが呼び出し側のスタックで動き、ioctl 経路の X25519 と割り込みフレームで約4.3KB > 4096。記録)、既定のスタックを大きくして直しました(QEMU で 20/20)。 *Say:* "No, it is the stock protected config: the kernel uses about eighty percent of its part, the user command about eight percent of the other. The intermittent fault was not the memory layout: the wg stack was too small. It is fixed, twenty out of twenty in QEMU."

Q. usrsock □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□?(□□□□29) SPRESENSE □□□□□□□□□□: □□□ usrsock □□□□□□□1□□□□□□□□□□□ □□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□2□□□ down □ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ IOB □□□ □□□□ Wi-Fi □□□□□□□□□□□□(sim □□ IOB □□□□□ PASS)□ *Say:* "I forced it on SPRESENSE: a query answered, a peer update was accepted, stop reported a timeout instead of hanging, and a second down cleaned up the stuck packet. Not covered yet: running out of IOBs on hardware, Wi-Fi loss mid-send, long heavy traffic."

Q. □□□□□□□□□□□□□□□□□□□□□?(□□□□30□sim) □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□ □□□1□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ UDP □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□down □□□□□□□□□□□□□□□□□(□□ TZ □□□□□)□ □□□□□□□□□□□□□□□□□□□□□□1□□□□□□ *Say:* "In the simulator. I found the device by searching for the key I had set, then checked each secret field by name, at offsets from the driver's own headers. To catch the handshake state I held back the last handshake message with a UDP relay."

Q. IOB(□□□□□□□□)□□□□□□□□□□□□? □□□□12□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□ — assert□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□IOB□□□□□□□□□□□□□□□□□□□□□ □□□ [iob-wait-audit.md] □□□ □□□□□: □□□□□□UDP□□□□□□□□□□□ try-allocation (`throttled=false` / `timeout=0`)□□□□`nwait` □□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□ — □□□□□ □□□□□□□□□□□□□usrsock□□□□□□□□□□□□□□□□□AP□□□□□□□□□□□□□□□□□□□□□□□□□ *Say:* "Shrinking the pool and flooding it survives — no assert, full recovery. Whether the driver can ever block waiting for a buffer is settled, not open: the direct TX/RX paths are try-allocation only, so there's no wait branch to exercise. That's 'not reachable,' which is a better answer than 'untested.'"

Q. #14(□□□□□□□/□□□□)□□□□□ open □□□□□□□ □□□□□□□?(□□□□31) □□□□□□□□□□□□□□□□□□□ open □□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ — □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□ hostfs □(POSIX □□□)□□□□ `fsync()` □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□ □□□□□□□□□□□□□: □□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□75□□□□□□□□□□□□□□□□□□ □□□□□□□□4.1□□□□□□ — □□□□□□□□□□□□□□□□□□ □□□□□□□ *Say:* "It's open by decision, not by lack of evidence. The durable answer needs a storage durability contract NuttX doesn't offer yet, so I won't ship something that only looks solved. I did measure the failure on real hardware with a controlled pair — clock unset, no handshake in 75 seconds; clock set, 4.1 seconds."

Q. □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□? □□□□□□□□75□□□□□□?(□□□□31) □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□1□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□75□□ □□□□□□□□□□□□□□□□□□□: □□□□□□□□□□□□□□□□□ □□□75□□□□□□□□□□□□□□□□□□□□□□□□□75□□□□□ □□ ping □□□□□□□□16□□□□□□□□□□ — ping □□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ *Say:* "A peer that meets this key for the first time accepts it, so refusing would break first pairing on exactly those boards. The 75 seconds in the simulator was only the length of the last session; on the board, 75 seconds is how long I watched. And a ping after reset passes in 16 seconds, because the peer starts the handshake."

Q. その「**TZ(**鍵のゼロ化**)**」の検証で **next_keypair** やハンドシェイク状態が 非ゼロになる瞬間は見られたのか**?** その後見られるようになりました。UDPリレーで転送確認を意図的に止めて `next_keypair`(responderが次のセッションをインストールしてから現行に 昇格するまでの短い窓)を非ゼロのまま捕捉し、応答のない開始要求で ハンドシェイク状態(鍵導出の瞬間に消去される一時鍵・チェイン・ハッシュ) も非ゼロのまま捕捉しました。どちらも `wg down` 後はゼロです。タイマー 駆動の自然な再鍵化 (ピア側のリセットなし)もこの間に確認しています。 *Say:* "It's closed now — a UDP relay that withholds the transport confirmation catches `next_keypair` non-zero, and an unanswered initiation catches the handshake state non-zero. Both go to zero after `down`, on sim, under one controlled delivery schedule."

Q. PROTECTED ビルドは問題なく動くのか**?** rv-virt の protected 構成で、`wg` をユーザー側に置いたままトンネルが Linux まで届くことは 確かめました。10/10 に繰り返し実行したところ、8回中2回ユーザー空間で例外が起きました (分割前の候補でも9回中1回)。原因は `wg` コマンドのタスクのスタックあふれでした。 PROTECTED ではシステムコールが呼び出し側のユーザースタックの上で動くので、4096 バイトの スタックに、コマンド、トラップフレーム、ドライバの ioctl(鍵や peer を設定するときは X25519 の計算)、その最中の割り込みのフレームが同時に乗り、約 4.3 KB でスタックの下の TLS ブロックを壊していました。割り込みが一番深いところで入ったときだけなので間欠的でした。 PROTECTED のときの既定のスタックを 6144 バイトにして、QEMU で 20回中20回成功しました。実機の PROTECTED では確かめていません。 *Say:* "The tunnel works with the MPU split on. In repeated runs, two out of eight first hit a fault. It was a stack overflow: in a PROTECTED build a system call runs on the caller's own stack, so the small wg stack also had to hold the driver's key math and an interrupt frame, and only sometimes it ran out. I made the default stack bigger for PROTECTED builds, and then twenty out of twenty runs passed. That is in QEMU; I have not run PROTECTED on real hardware."

Q. 生成 **AI** の時代に、なぜ **NuttX** なのか**?** AI が持っている知識の量では FreeRTOS や Zephyr が上ですが、NuttX は POSIX なので、AI が いちばんよく知っている Linux の書き方がそのまま効きます。sim と QEMU で実機なしに 「ビルド → 実行 → 確認」を回せ、設定も操作 (NSH)もテキストなので、AI エージェントが 自分で試して直せます。この移植もその形で進めました。一方で、学習データが少ないぶん、 AI は NuttX にない機能をもっともらしく作ることがあるので、試験で確かめることが前提です 。Zephyr にも native_sim と twister があるので、2つ目の 点は互角です。 *Say:* "NuttX speaks POSIX — the language AI already knows best. Most of what an AI knows about Linux code just works here, and with sim and QEMU it can build, run and check without hardware. But it also means the AI will sometimes invent things NuttX does not have, so you still need deep tests."

---

upstream・提出状況

Q. もう**PR**は出したのか**?** マージされているのか**?** まだ何も送っていません。マージもされていません。作業は CONTRIBUTING の規則に合わせて 手元で小さな単位(crypto 修正 → nxstyle の除外 → ドライバ → `wg` コマンドと文書)に分け、 コミットごとに単独でビルドと検査が通る状態にしてあります。 **PR**の作成も **dev@nuttx.apache.org** への投稿もしていません。 コミットにはまだ署名 (Signed-off-by)がなく、最終的なシリーズは実機で動かしていません。この場で示しているのは 「こう作った、こう検証した」という今の状態です。 *Say:* "Nothing is sent, and nothing is merged. The work is split locally into small pieces, and every commit builds and passes the checks on its own. The commits are not signed off, and the final series has not run on real hardware."

Q. checkpatch や **nxstyle** は通っているのか**?** はい。ファイル単位の `nxstyle -f` に加え、CIがPRに対して行うパッチ形式の **`checkpatch.sh -c -u -m -g`** を **PR** に分けた全**12**コミットで実施し、すべて通っています。 本家 CI と同じ `-Werror` での

ビルドも、コミットごとに通ることを確かめました。 *Say:* "Yes. Per-file nxstyle and the patch-form `checkpatch.sh` pass on every commit of the split series, and each commit builds on its own with -Werror."

Q. これは**GSoC**プロジェクトだったのか**?** 出発点にはGSoC提案としての検討もありましたが、この発表で示すコードと検証は、 採否とは切り離して継続したコミュニティ向けの開発成果です。だから発表の軸も 制度ではなく、実装、失敗、検証、そして作業が今どこにあるかに置いています。 *Say:* "It began with work around a GSoC proposal, but the engineering continued independently. This talk is about the implementation, the evidence, and where the work stands, not the program outcome."

Q. NuttX自身のバグ**(ChaCha20-Poly1305**の**nonce)**は誰が直すのか**?** このプロジェクトの成果に含まれるのか**?** このプロジェクトの中で見つけたバグですが、直し方は独立したものです。 □ウンタ0では新旧のnonceレイアウトが一致するため、これまで誰も (in-treeの呼び出し元もなく)踏んだことがありませんでした。修正は 2行で、ドライバとは別の単独の `crypto:` の変更として分けてありますㇲ — それ単体で 成り立ち、ドライバがなくても役に立つからです。まだ送ってはいません。 *Say:* "Found while porting, but it's a genuine pre-existing NuttX bug, not something the driver introduced. Two-line fix, kept as its own `crypto:` change, apart from the driver. It is not sent yet."

性能・運用

Q. 性能は**?** これは **apps** 版**(v0.1.1ㇾ FLAT)**の数字です。 ESP32-S3 実機・実 Wi-Fi 経由で、トンネル越しの TCP スループットが約 260 KiB/s です。7 MB の連続転送を問題なく通せています。ただしこれは 「明らかなボトルネックを 1つ潰した」段階の数字で、体系的なチューニングは していません。暗号処理がソフトウェア実装のままなので、そこがおそらく 次のボトルネックです。 □ーネルドライバのスループットはまだ測っていません (SPRESENSE での測定は ICMP の負荷で、スループットではありません)。 *Say:* "That number is for the apps version: about 260 KiB/s TCP through the tunnel on ESP32-S3 over real Wi-Fi, 7 MB transfers clean. That's after fixing one obvious bottleneck, not a tuned number — software crypto is probably next. The kernel driver's throughput is not measured yet."

Q. 暗号処理はソフトウェア**?** ハードウェアアクセラレータは**?** 現状はNuttX自身のソフトウェア実装(BLAKE2s, ChaCha20-Poly1305, Curve25519)をそのまま使っています。cryptodev経由でCurve25519に 到達すること自体は技術的に可能です (`CRK_DH_COMPUTE_KEY` → `swcr_dh_make_common`)が、そのハンドラは `CRYPTO_CRYPTODEV_SOFTWARE_CRYPTO` 経由でしか有効にならず、それは `CRYPTO_SW_AES` に依存しています。 つまり 1回のス□ラー倍算のために、ソフトウェア暗号スイート一式を イメージに引き込むことになり、割に合わないと判断して見送りました。 *Say:* "Software throughout. Reaching curve25519 via cryptodev is technically possible but drags in the whole software cipher suite as a dependency price — not worth it for one scalar multiplication."

Q. 長時間の安定性は**?** apps版(ESP32-S3)では最長4時間28分の連続稼働を確認していて、止まった 原因は ESP32-S3のWi-Fiドライバ側の問題でWireGuardとは無関係でした。 □ーネル版については、sim上で151回の強制再鍵化・10回のエンドポイント 移動・100回のdown/upを620秒で回し、リソースが起動時と完全に一致する ことを確認していますが、これはシミュレータです。実機側はSPRESENSEで120秒の負荷と40回のdown/upサイクルを確認済みで、こちらも 資源は前後で完全一致します。ただし「日単位で動く」ことはまだ実機の □ーネル版では言えません。 *Say:* "The apps version ran 4h28m on hardware before an unrelated Wi-Fi driver bug stopped it. The kernel driver's soak is 620 seconds of forced churn in the simulator, plus 40 down/up cycles measured on real SPRESENSE hardware — both come back to identical resource numbers. Day-scale uptime on the kernel driver, on hardware, isn't something I can claim yet."

Q. 複数ピアは**?** 対応済みです。 `CONFIG_NET_WIREGUARD_MAX_PEERS` で1〜16まで設定でき、 シミュレータ上でLinux□ーネルのWireGuardを2本立てて、両方と同時に セッションを保持することを確認しています。ピアは静的に確保されるので RAMと直接の

トレードで、実測で1ピアあたり約904バイトです。実機での 複数ピア確認はまだです。 *Say:* "Supported, 1 to 16 via Kconfig, verified with two simultaneous Linux peers in the simulator — about 904 bytes of RAM per peer, statically allocated. Not yet confirmed with multiple peers on hardware."

Q. 鍵の管理は**?** 実行時に設定できます。`wg genkey` で生成し、`wg set private-key` で 設定、`wg saveconf` で保存すると次回起動時に自動で読み込まれます。 ビルド成果物に鍵を焼き込む必要はありません。設定ファイルの二重 書き込みで壊れるバグ (#17) を2件見つけて直しました — 同時に2つの 書き込みが起きると設定が消える経路があったのと、共有の一時ファイル名が 競合する経路です。SmartFS上での電源断耐性はまだ未検証です (切り替え可能な電源が必要)。 *Say:* "Runtime genkey/set/saveconf, nothing baked into the build. Found and fixed two real bugs in the config-save path where concurrent writers could destroy the file. Power-cut durability on SmartFS specifically is still unverified — needs switchable power I didn't have this cycle."

Q. 設定ファイルの形式は独自**?** `wg(8)` と同じ INI 形式です(`[Interface]` / `[Peer]`、`PrivateKey` / `PublicKey` / `AllowedIPs` / `Endpoint` / `PersistentKeepalive`)。デスクトップの `.conf` をそのまま持ち込めます。 *Say:* "Same INI format as `wg(8)` — a desktop `.conf` drops in as-is."

ハードウェア選定

Q. なぜ **ESP32-S3** と **SPRESENSE** を選んだのか**?** NuttXのWi-Fiサポートがあることが第一です。手元のESP32-WROOM-32は 書き込みに入れず(GPIO0のブートモード遷移まで切り分け済み)、SPRESENSEは オンボードWi-Fiがなく外付けのGS2200Mモジュール経由になるので、 usrsock という別の経路(ボード側の `socket()` がWi-Fiモジュールに 渡る)も同時に検証できる利点がありました。 *Say:* "NuttX Wi-Fi support was the filter. SPRESENSE's Wi-Fi is an off-chip GS2200M module through usrsock, which is a genuinely different code path from ESP32-S3's native Wi-Fi — so the two boards together cover two different integration styles, not just two chips."

Q. Raspberry Pi Pico 2 W は使えないのか**?** オンボードのWi-Fiチップ(CYW43439)のドライバが、NuttXではRP2040版にしか統合されていません。RP2350側にもPIO/GPIOの対応関数は揃っている ので移植自体は現実的ですが、別のドライバ開発タスクになるので今回は スコープ外としました。 *Say:* "The onboard Wi-Fi chip's driver is only wired up for RP2040 in NuttX today, not RP2350 — porting it is plausible but a separate driver task, out of scope here."

バージョン依存

Q. NuttXのバージョンは何に依存しているのか**? apps** 版はしていません。12.7.0とmasterの両方でビルドが通ります (`--build-arg NUTTX_REF=<ref>`)。masterでは `CONFIG_NSH_LINELEN` が `CONFIG_LINE_MAX` に改名されており、古い名前を指定してもエラーに ならないまま `wg set peer` の行だけが切れる、という静かな壊れ方を 一度見つけて直しています。 カーネルドライバは別です**:** NuttX 本体の小さな変更2つ(netdev upper half と ioctl 番号)が前提で、提出シリーズは master の上に載っています。 *Say:* "The apps version builds clean on both 12.7.0 and master via a build arg. Found one silent breakage along the way — a renamed Kconfig symbol that truncated a config line without erroring — and fixed it. The kernel driver is different: it needs two small changes in the network core, and the series is on master."

台本と合わせて使う補足

- 質問が来る可能性が高いもの: upstream提出状況(fork push済み、PR/dev@は未実施)、ESP32-S3の資源実測待ち、#17のSmartFS電源断待ち。 すべてこの資料で答えられるようにしてある。 TZ・TIはどちらも もう閉じている(前者は測定、後者は「到達不能」の確認) — 古い 資料や記憶で「まだ狭い」と言わないこと。
- 逆にスライドに載っている数字を聞かれたら、スライド29・30・31を 直接指させばよい(URLハッシュ `#18` などで即座に飛べる)。